ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

Αρχές Σχεδίασης

Ποιότητα Λογισμικού

Απόστολος Αμπατζόγλου
Επίκουρος Καθηγητής
a.ampatzoglou@uom.edu.gr

Outline

Responsibility Assignment Patterns

Object-Oriented Design Principles

Mastering a Foreign Language

- First learn the rules
  - syntax, grammar, irregular verbs etc.
- Then learn the principles
  - Expressions for different situations, written vs. spoken form, etc.
- But to really master a foreign language
  - study others speaking or writing
- Every language has thousands of these patterns

An example

- **Context:** Arranging a living room
- **Forces:**
  - People love window seats, big windows and comfortable chairs.
  - A room without such places does not make us feel at ease.
  - We want to sit down and be comfortable
  - We are drawn towards the light

A window place
(Alexander et al. 79)

- **Therefore:** in every room where you spend any length of time during the day, make at least one window into a "window place"

Introduction

Important Decisions:

- Where should I place a method?
- How do objects interact?

A problem in the heart of OO

Is there any guidance?

Design Principles + Software Patterns

Two types of objects responsibilities:

Actions

- Sole responsibilities (e.g., object creation, calculation)
- Initiate an action in other classes
- Control actions of other classes

Knowledge

- Private data
- Related objects
- What it can produce or calculate

Outline

Responsibility Assignment Patterns

- Information Expert
- Creator
- High Cohesion
- Low Coupling
- Controller
- Polymorphism

Object-Oriented Design Principles

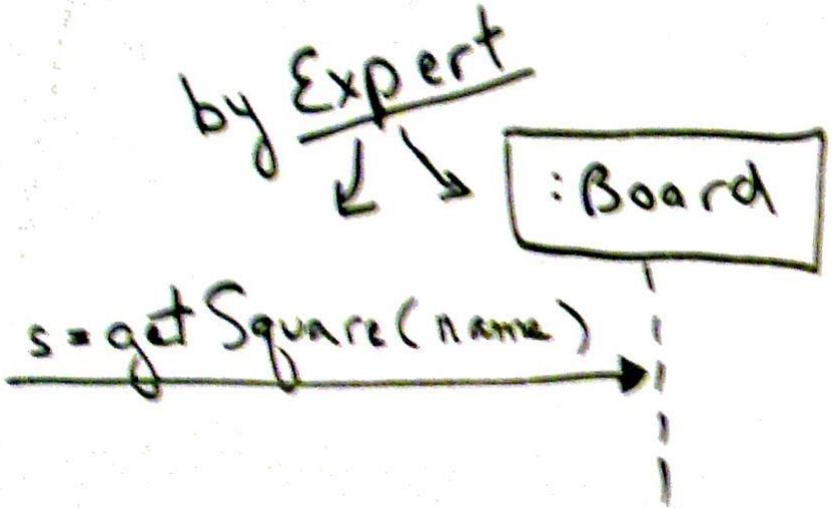
Information Expert

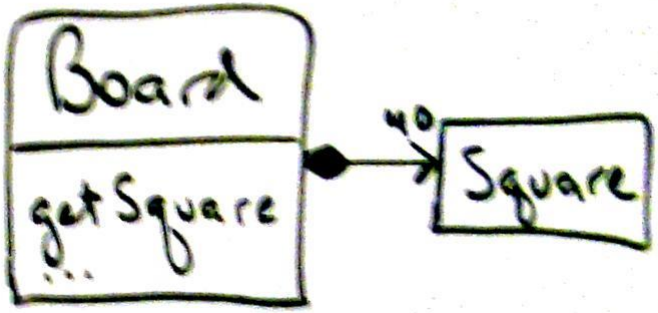
Name: Information Expert

Problem: How to assign responsibilities to objects?

Solution: Assign responsibility to the class that has the information needed to fulfill it?

e.g., Board information needed to get a Square





Facilitates information encapsulation: why?

Classes use their own info to fulfill tasks

Encourages cohesive, lightweight class definitions

Information expert may contradict patterns of Low Coupling and High Cohesion

Remember separation of concerns principle for large sub-systems - keep “business” or application logic in one place, user interface in other place, database access in another place, etc.

Outline

Responsibility Assignment Patterns

- Information Expert
- Creator
- High Cohesion
- Low Coupling
- Controller
- Polymorphism

Object-Oriented Design Principles

Creator Pattern

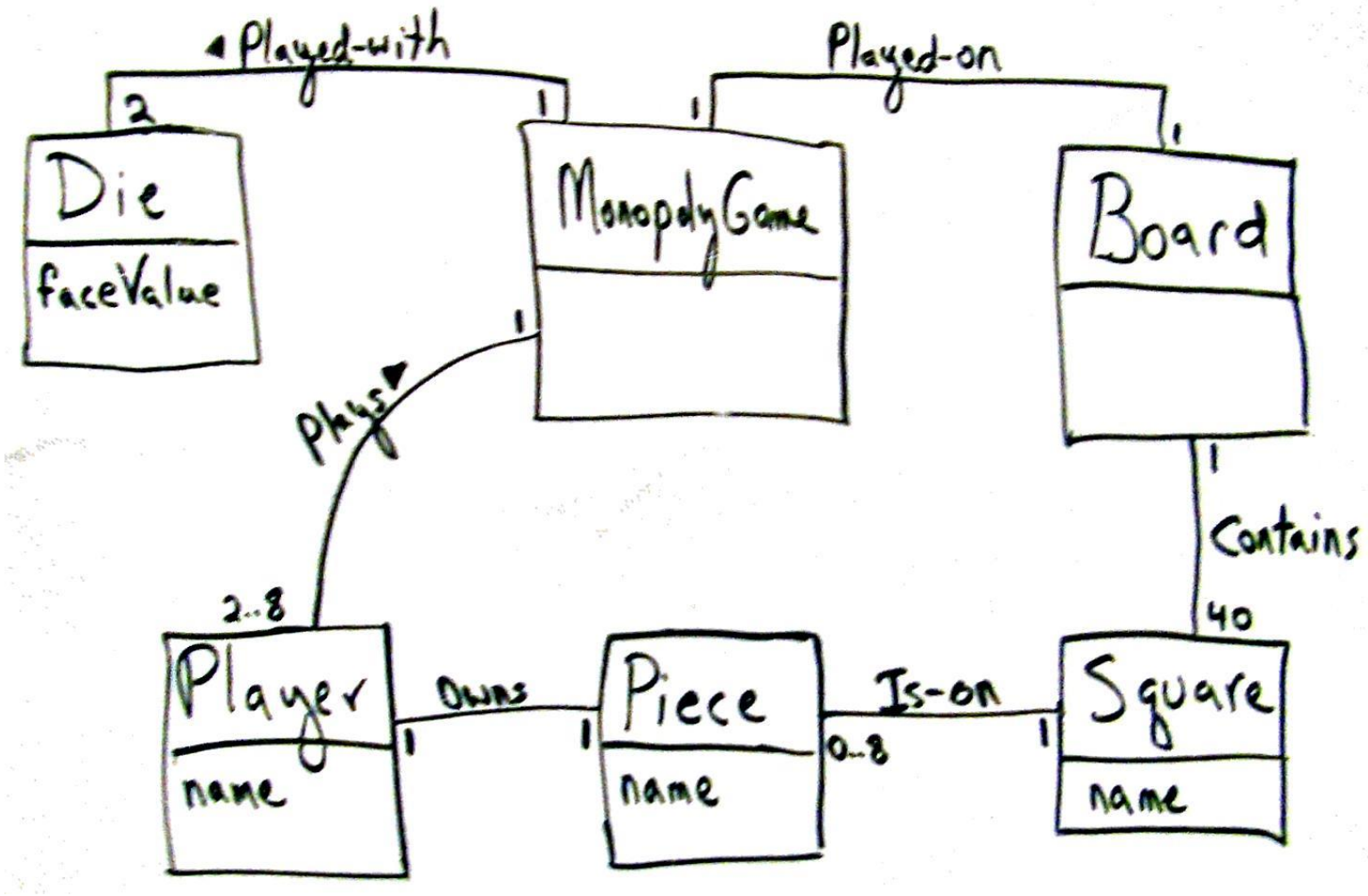
Name: Creator

Problem: Who creates an instance of A?

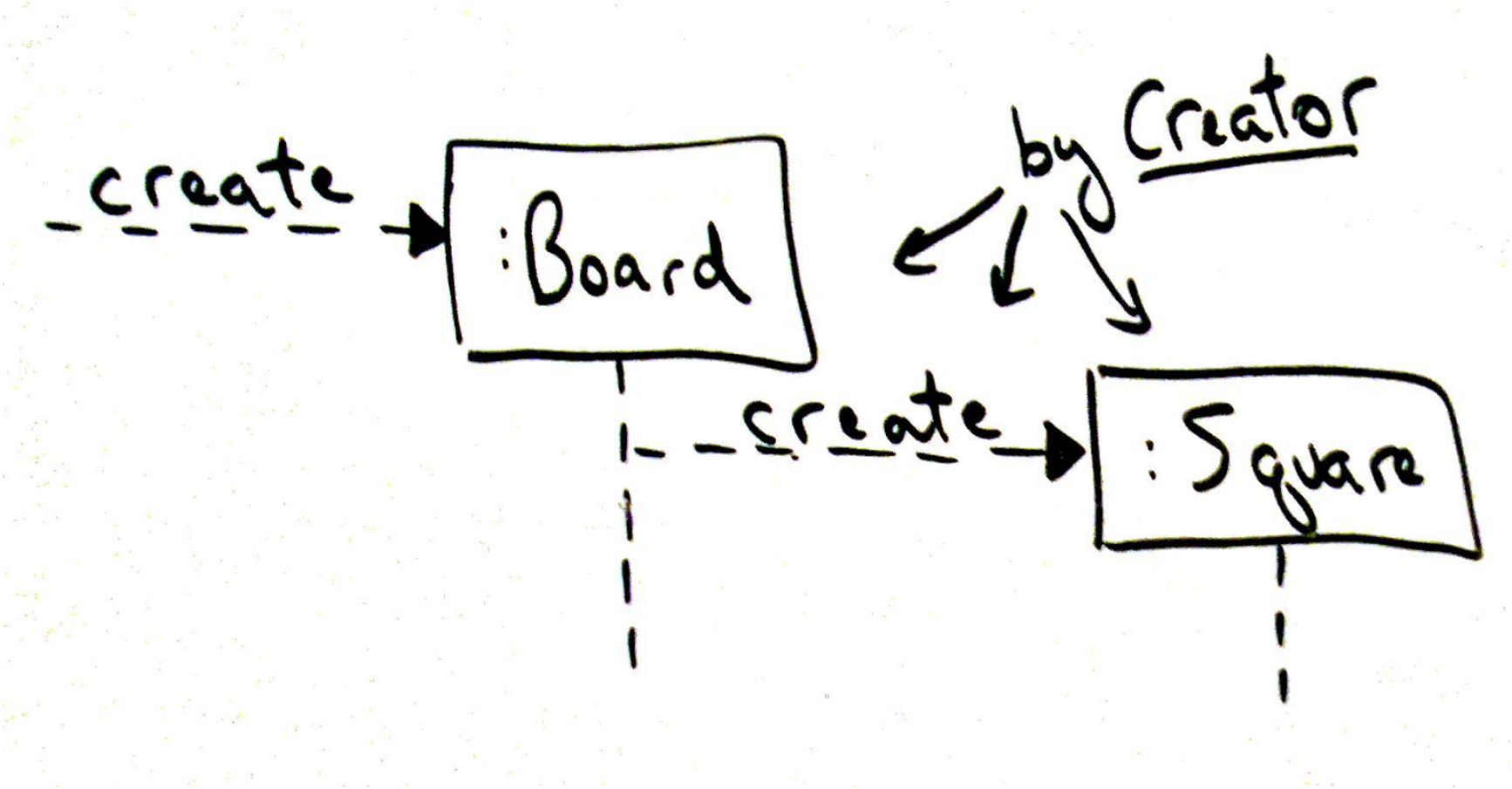
Solution: Assign class B the responsibility to create an instance of class A if one of these is true (the more the better):

- B contains or aggregates A (in a collection)
- B records A
- B closely uses A
- B has the initializing data for A

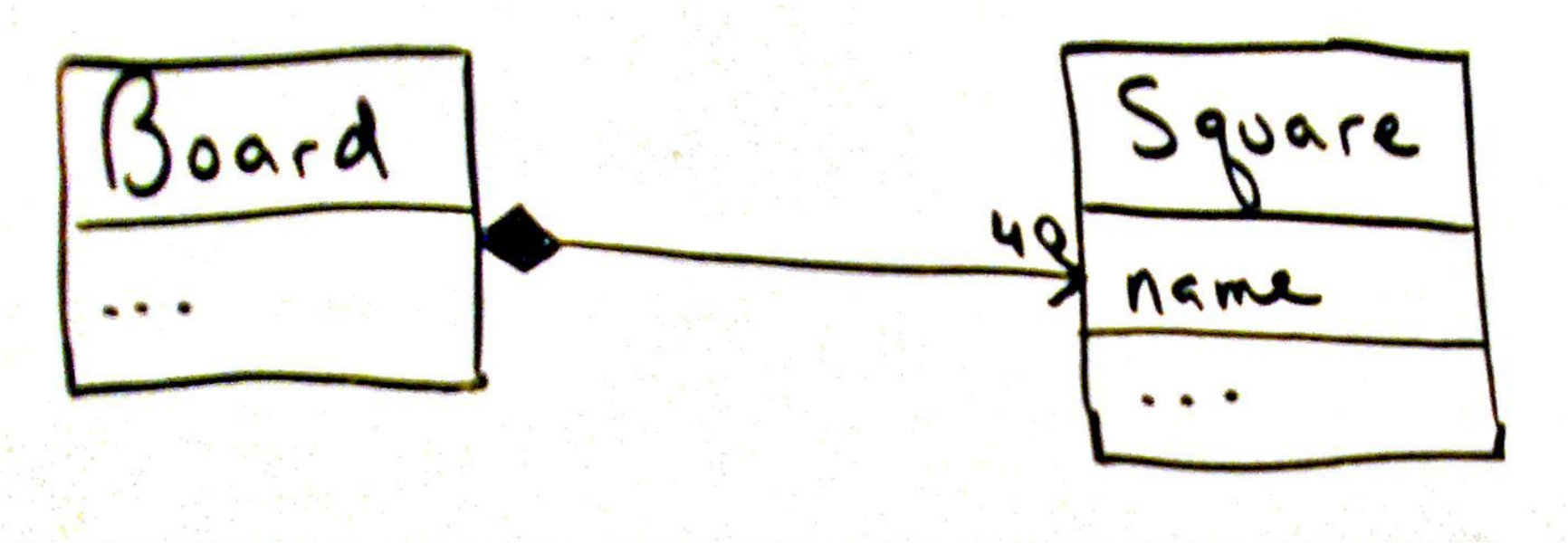
Who creates the squares?



Sequence Diagram



Class Diagram



Board has a composite aggregation relationship with Square

- i.e., Board contains a collection of Squares

Responsibilities for object creation are common

Connect an object to its creator when:

- Aggregator aggregates Part
- Container contains Content
- Recorder records
- Initializing data passed in during creation

Creation may require significant complexity:

- recycling instances for performance reasons
- conditionally creating instances from a family of similar classes

In these instances, other patterns are available…

- e.g., GoF Factory patterns

Outline

Responsibility Assignment Patterns

- Information Expert
- Creator
- High Cohesion
- Low Coupling
- Controller
- Polymorphism

Object-Oriented Design Principles

High Cohesion

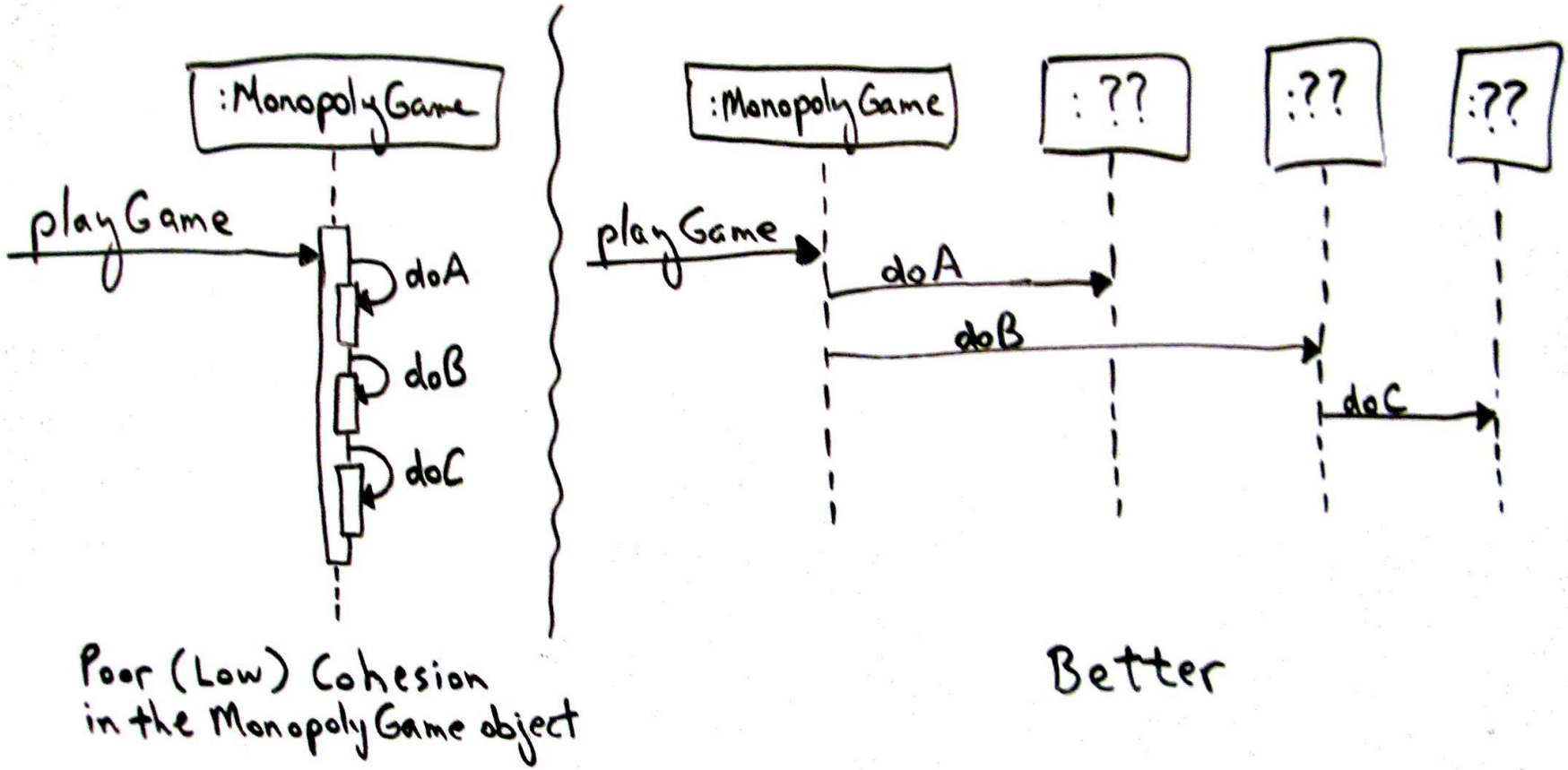
Cohesion measures how strongly related and focused are the responsibilities of an element

Name: High Cohesion

Problem: How to keep classes focused and manageable?

Solution: Assign responsibility so that cohesion remains high

Conformance



Delegate responsibility & coordinate work

Understandability, maintainability

Complements Low Coupling

Avoid grouping of responsibilities or code into one class or component to simplify maintenance by one person. Why?

Desirable to create less cohesive server objects that provide an interface for many operations, due to performance needs associated with remote objects and remote communication

Outline

Responsibility Assignment Patterns

- Information Expert
- Creator
- High Cohesion
- Low Coupling
- Controller
- Polymorphism

Object-Oriented Design Principles

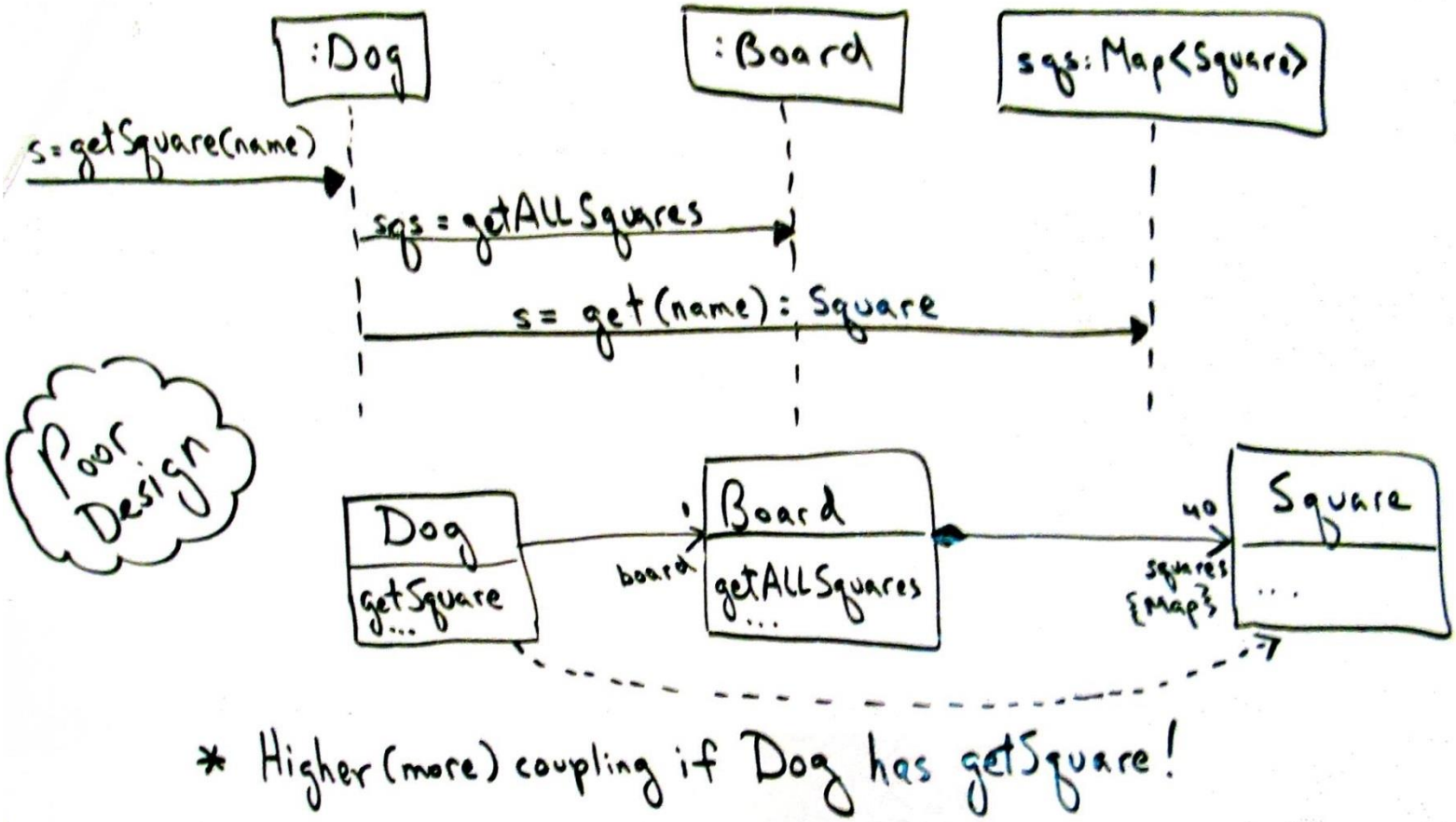
Low Coupling

Name: Low Coupling

Problem: How to reduce the impact of change and encourage reuse?

Solution: Assign a responsibility so that coupling (linking classes) remains low

Coupling Violation



Why is it better to leave `getSquare` responsibility in Board?

Understandability: Classes are easier to understand in isolation

Maintainability: Classes aren't affected by changes in other components

Reusability: easier to grab hold of classes

Don't sweat coupling to stable classes (in libraries or pervasive, well-tested classes)

Outline

Responsibility Assignment Patterns

- Information Expert
- Creator
- High Cohesion
- Low Coupling
- Controller
- Polymorphism

Object-Oriented Design Principles

Controller

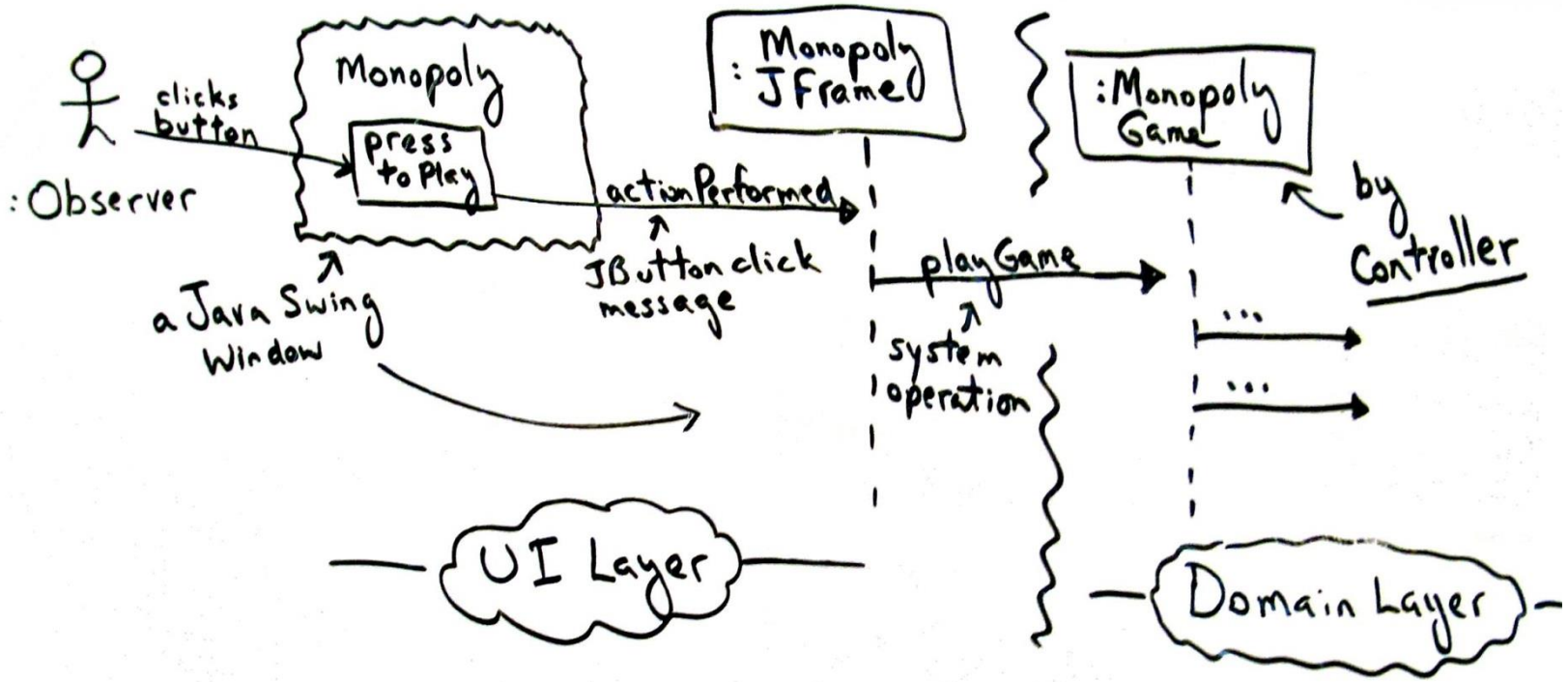
Name: Controller (see MVC architecture)

Problem: Who should be responsible for UI events?

Solution: Assign responsibility for receiving or handling a system event in one of two ways

- Represent the overall system (façade pattern)
- Represent a use case scenario within which the system event occurs (a session controller)

Who is the controller?



What class represents the overall system or relevant use case scenario?

Outline

Responsibility Assignment Patterns

- Information Expert
- Creator
- High Cohesion
- Low Coupling
- Controller
- Polymorphism

Object-Oriented Design Principles

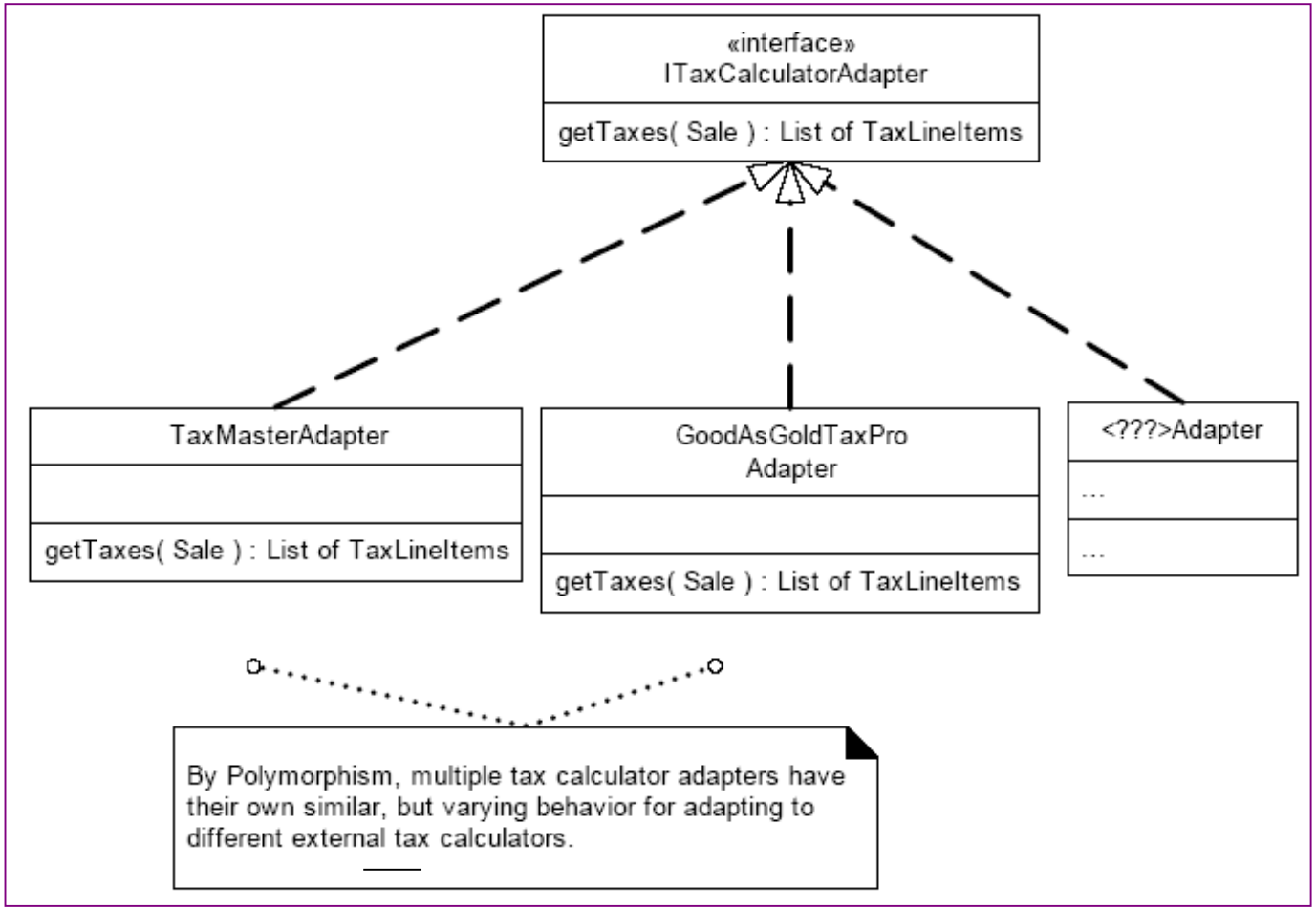
Polymorphism

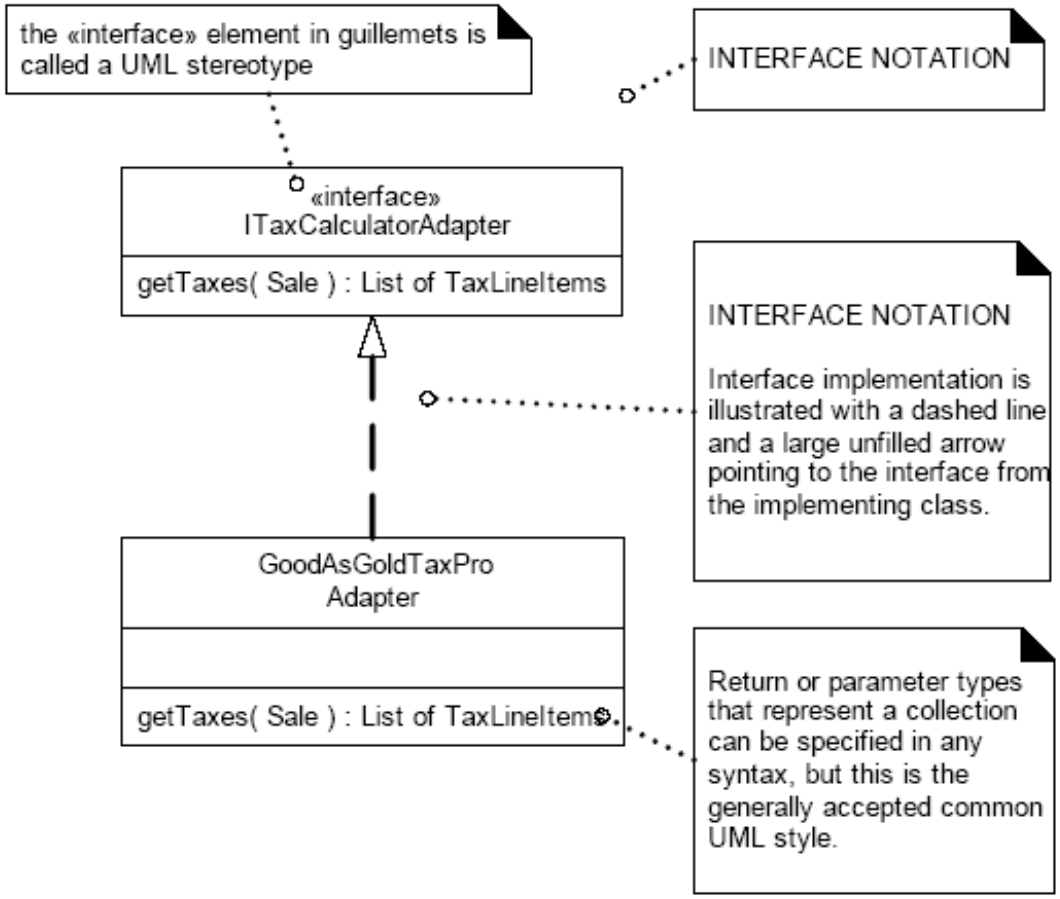
Solution when alternative behaviors are different in each type (class)

How to handle the alternative behaviors based on type? How to create run-time adaptable software modules?

Consider a POS: Different tax calculators, with different interfacing and protocols (TCP socket, SOAP, Java RMI)

The adapters are internal objects that are mapped to external tax calculators


«interface»
ITaxCalculatorAdapter
getTaxes(Sale) : List of TaxLineItems
TaxMasterAdapter
getTaxes(Sale) : List of TaxLineItems
GoodAsGoldTaxPro
Adapter
getTaxes(Sale) : List of TaxLineItems
<???>Adapter
...
...
By Polymorphism, multiple tax calculator adapters have their own similar, but varying behavior for adapting to different external tax calculators.


the «interface» element in guillemets is called a UML stereotype
INTERFACE NOTATION
«interface»
ITaxCalculatorAdapter
getTaxes(Sale) : List of TaxLineItems
INTERFACE NOTATION
Interface implementation is illustrated with a dashed line and a large unfilled arrow pointing to the interface from the implementing class.
GoodAsGoldTaxPro
Adapter
getTaxes(Sale) : List of TaxLineItems
Return or parameter types that represent a collection can be specified in any syntax, but this is the generally accepted common UML style.

Adaptability / Extendibility

The clients are not affected by adding/removing service providers

Poor performance

Outline

Responsibility Assignment Patterns

Object-Oriented Design Principles

Design Bad Smells

Rigidity: The system is hard to extend, because every change has heavy impact

Fragility: Changes in one part of the system causes defects in other parts of the system

Immobility: Difficulty to decompose the system to components that can be reused in different software.

Viscosity: System is more prone to extension through not optimum extensions mechanism than optimum ones.

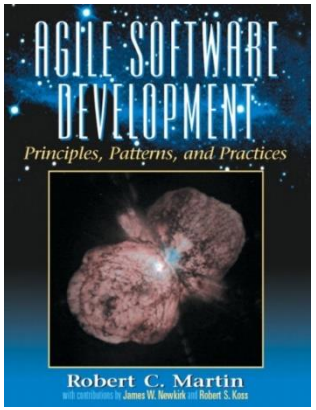
Needless Complexity: The software includes components that are not or will never become useful.

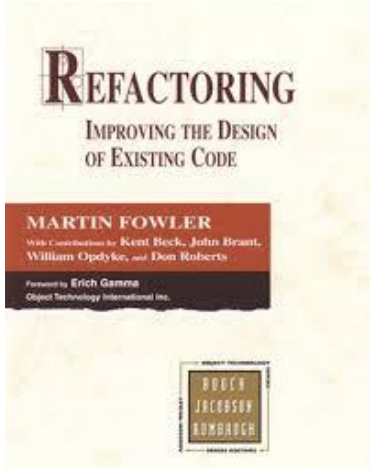
Needless Repetition: The design involves repeated structures that could be merged under a common abstraction.

Opacity: Difficulty in understanding a software unit (either code or design level).

Solution

- Object-Oriented Principles
- Software Refactorings
  - Definition
  - Many types, most known resource the Fowler refactorings





Outline

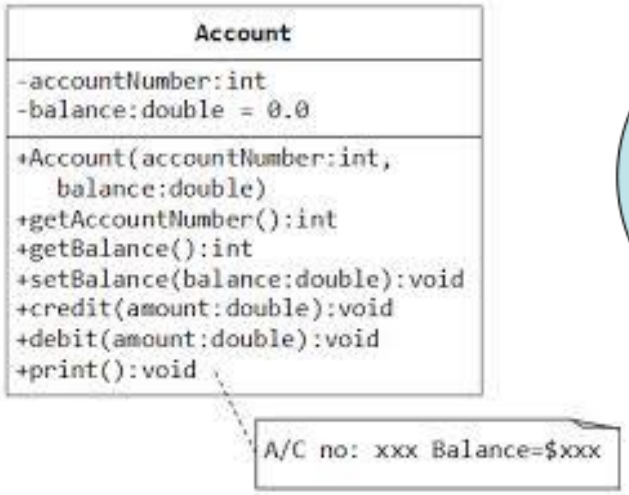
Responsibility Assignment Patterns

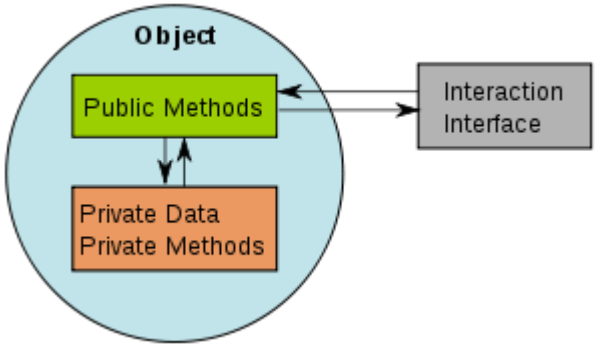
Object-Oriented Design Principles

- Encapsulation Principle
- Low Coupling Principle
- Single Responsibility Principle
- Open-Closed Principle
- Dependency Inversion Principle

Encapsulation Principle

- Basic OO principle
- Most of the times is used unintentionally
- The internal state of an object must be modifiable only through the public interface of the object





In practice you just have to set the visibility of all attributes to private

- Advantage: The validity of the object is persistent
- Each object has some constraint that always have to be valid
- Suppose a class `TimeStamp`

```
class TimeStamp {
public:
   TimeStamp();
   TimeStamp(int hr, int min, int sec);
   void printTimeStamp();

   int hour;                //μέλη δεδομένων με δημόσια ορατότητα !!!
   int minute;
   int second;
};
```

Construct an object: `TimeStamp T1(23, 45, 17);`

Suppose a client class of `TimeStamp` that increases by two hours the current timestamp, as follows:

```
//κώδικας κλάσης - πελάτη
   . . .
   T1.hour++;
   T1.hour++;
```

This will lead to a not-valid object (time: `25:45:17`)

Constraint { `0 <= hour <= 23` }

By applying the encapsulation principle, the designer can guarantee the validity of all objects of the class

- Design a constructor that instantiates only valid objects
- Design methods that are not allow the violation of constraint

```
class TimeStamp {
. . .
public:
   void incrementHour();

private:
   int hour;
   int minute;
   int second;
. . .
};

void TimeStamp::incrementHour() {
  hour++;
  if(hour == 24)
    hour = 0;
}
```

- Violating this principle can lead to severe problems

- By allowing the modification of attributes from objects outside the class ruins the clustering of data and behavior

- Severe testing/debugging problem, in the sense that there will be uncertainty on which parts of the system alter which data

Outline

Responsibility Assignment Patterns

Object-Oriented Design Principles

- Encapsulation Principle
- Low Coupling Principle
- Single Responsibility Principle
- Open-Closed Principle
- Dependency Inversion Principle

Low Coupling Principle

Object-Oriented Units: Independent Software Components

- with clear functionality
- with clear input/output sets

Basic terms in software engineering:

- Coupling: Dependencies between components
- Cohesion: Degree of internal functional relevance/similarity between parts of a component

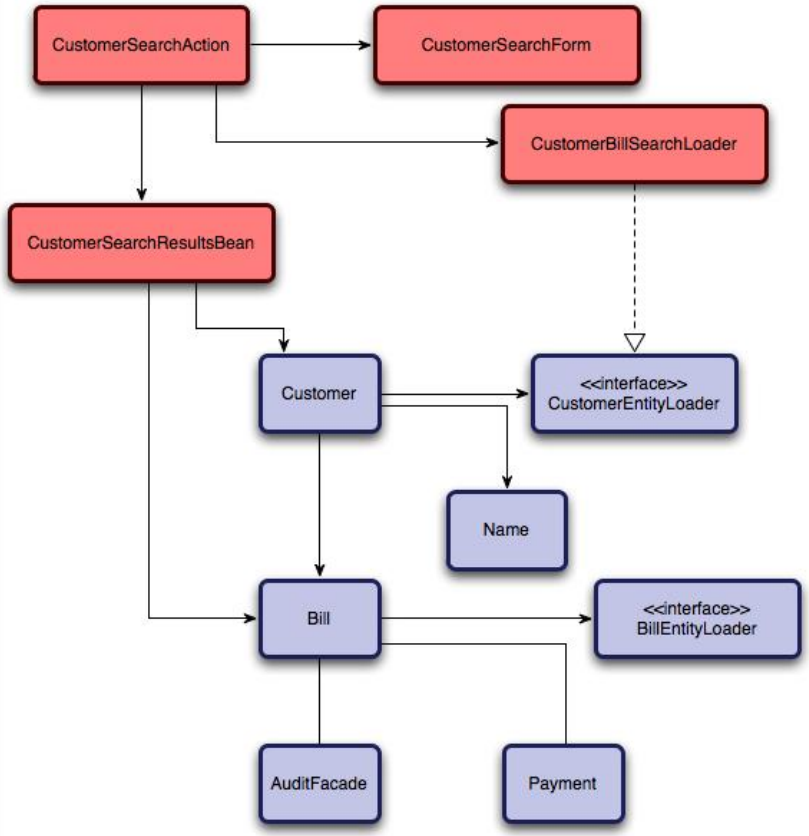
Principle: In a software design coupling should be as low as possible

Advantage:

- Easier development, testing, and maintenance

An example to avoid…

- if Customer changes we need to re-test
- and re-compile all dependent classes
- if we want to reuse Customer we need
- to reuse all dependent classes



Outline

Responsibility Assignment Patterns

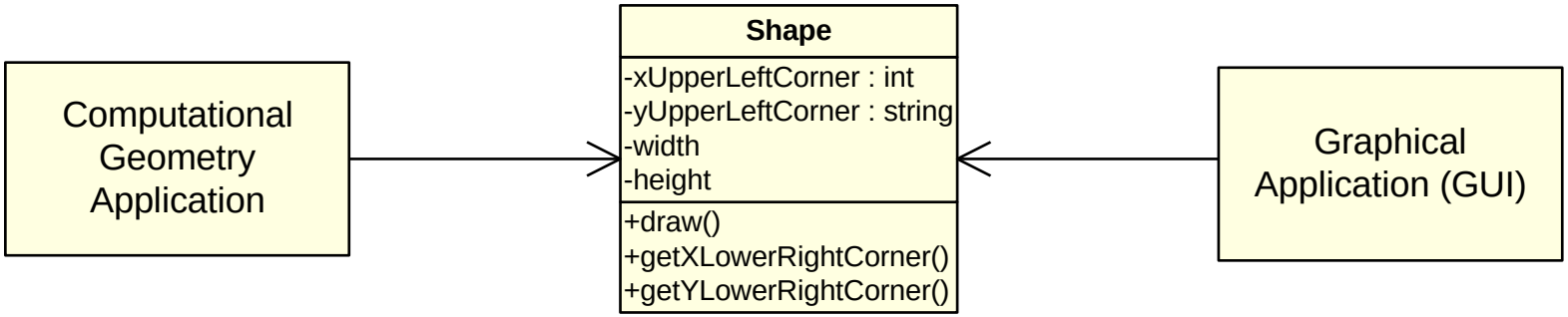
Object-Oriented Design Principles

- Encapsulation Principle
- Low Coupling Principle
- Single Responsibility Principle
- Open-Closed Principle
- Dependency Inversion Principle

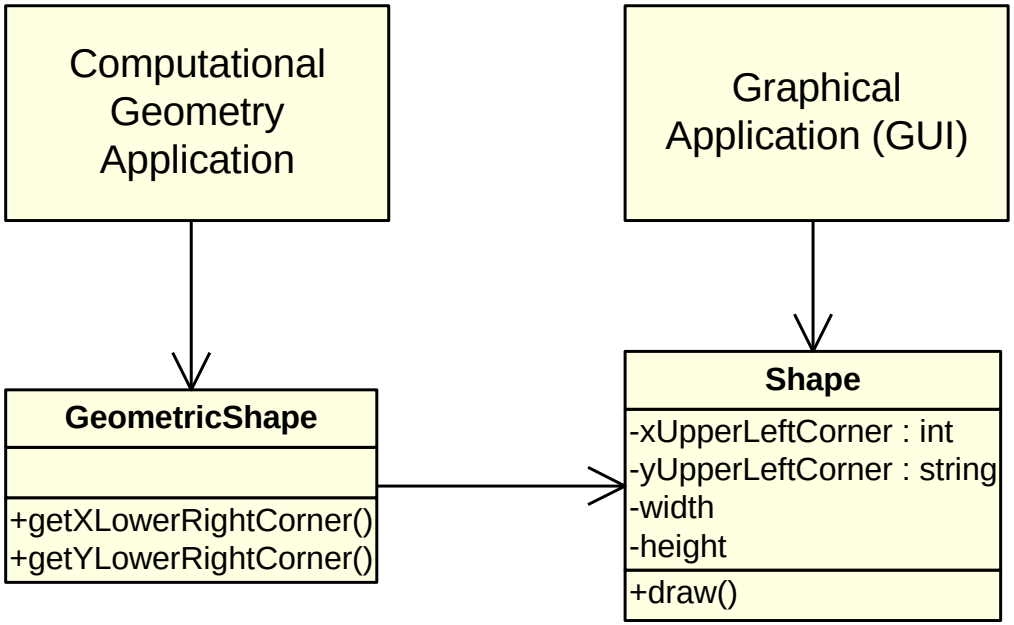
Single Responsibility Principle

- Principle: One class must have only one reason to change
- Each class must NOT have more than one responsibilities
  - more reasons to change coupling between responsibilities

- Suppose the following design:



Applying SRP impose dividing the two responsibilities into different classes



What is responsibility?

In the context of SRP, responsibility is defined as an "axis of change"

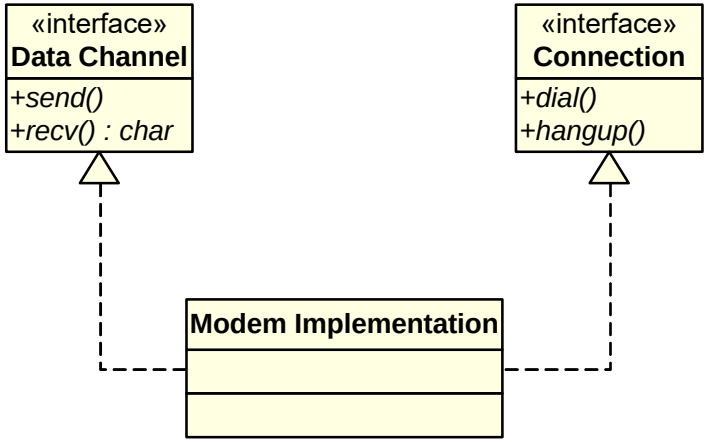
```
interface Modem
{
  public void dial(String pno);
  public void hangup();
  public void send(char c);
  public char recv();
}
```

Here we have two responsibilities. Should we split the class?

It depends on how the application changes....

Change frequently in a way that the signature of connect methods is changing, then we might observe some rigidity since classes that call methods on communication (`send` and `recv`) should be re-compiled.

In this case, the two responsibilities should be split. This way, application that use Data Channel interface, or the Connection interface are decoupled.



On the contrary, if the application does not change in a way that the two responsibilities are independently changing, there is no need for class splitting

In this case splitting the class would cause additional complexity

Conclusions:

- SRP is a simple principle. However, is really hard to apply
- Combining responsibilities is a natural way of thinking
- Spotting and splitting responsibilities is a substantial part of software design

Outline

Responsibility Assignment Patterns

Object-Oriented Design Principles

- Encapsulation Principle
- Low Coupling Principle
- Single Responsibility Principle
- Open-Closed Principle
- Dependency Inversion Principle

Open-Closed Principle

- "All systems change during their lifecycle"
  - Only three truths about life: Death, Taxes and Requirement Changes !!

- Principle: Software units (classes, components, methods etc.) must be open for extension and closed for modification

- If OCP is correctly applied → requirements or changes of the same type → adding new code → not by modifying existing code
  - Open for extension: The behavior of the module can be expanded
  - Close for modifying: Such extensions do not lead to changes on source or binary code of the unit.

Abstraction is the key

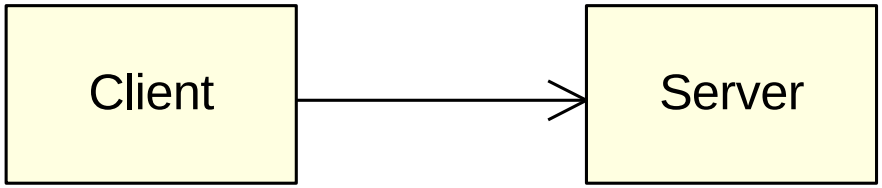
In every OOPL it is do-able to construct abstractions that are stable and defined. At the same time they represent an unlimited number of behaviors

A module can handle this abstraction

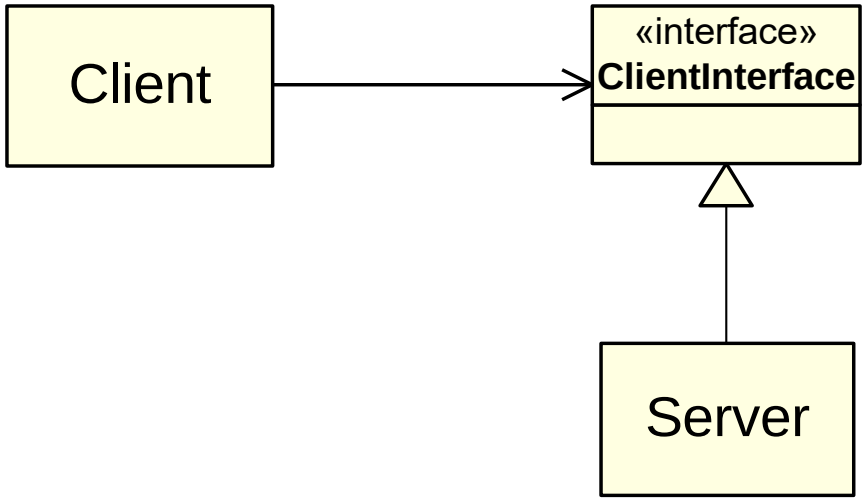
A module that handles an abstraction is closed to modifications, since it is dependent to the είναι abstraction which is stable.

However, the behavior of the module can be extended by adding sub-classes in a hierarchy.

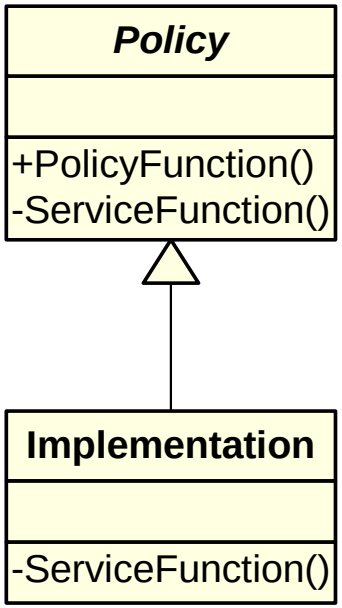
Violation of OCP



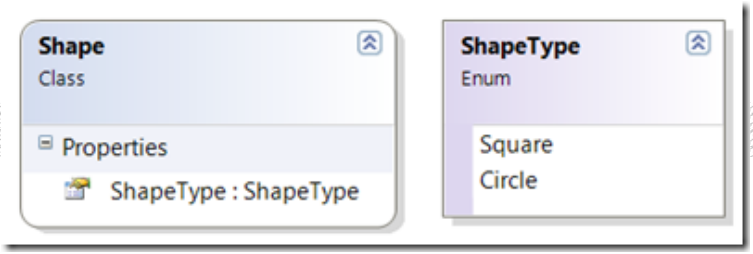
Conformance to the OCP (Strategy pattern)



Alternative Structure (Template Method pattern)



These two patterns are the most common way to comply with OCP



Shape Application:

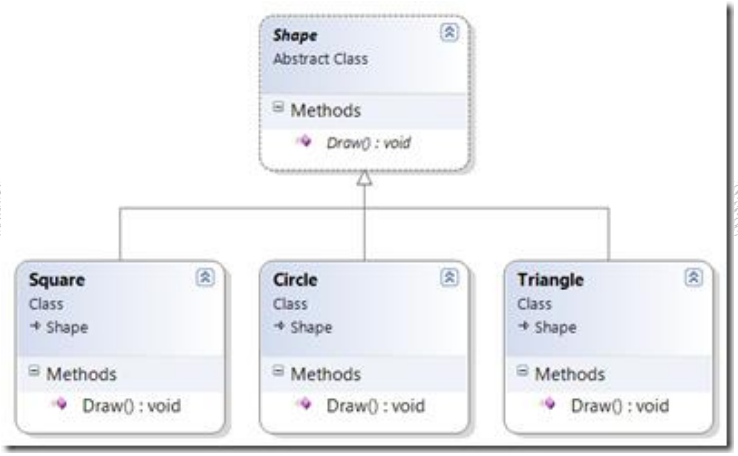
Suppose a Painter that must draw circles and rectangles in a GUI. Another application (e.g. class `ShapeApplication`) creates a list of circles and rectangles and must draw each shape.

Suring run-time the type of the object is explored.

- Does class `Painter` conforms to OCP ? (among others Painter is involved in “foreign responsibilities”)
- In practice we need to identify any possible switch and `if/else` structures (move shape, resizing, delete, change color)
- Usually `switch` structures involve logic operators

Bad Design:

- Rigidity: All modules that are depended on Painter must be recompiled or tested in case that a new shape should be handled (e.g. Triangle). As a ripple effect all related files need to be recompiled
- Fragility: Such an addition requires the identification, understanding, and modification of all the corresponding switch and if/else statements
- Immobility: In order to reuse Painter you need to reuse other files



Improved Design (OCP)

Rigidity: Eliminated. No additional files need to be re-compiled after adding a new shape

Fragility: Eliminated. No need to identify conditional statements

Immobility: Eliminated. Painter can be reused without concrete subclasses

Modified by adding new subclasses and not by modifying existing code

Outline

Responsibility Assignment Patterns

Object-Oriented Design Principles

- Encapsulation Principle
- Low Coupling Principle
- Single Responsibility Principle
- Open-Closed Principle
- Dependency Inversion Principle

Dependency Inversion Principle

a. High level modules are not allowed to depend on modules of lower levels.

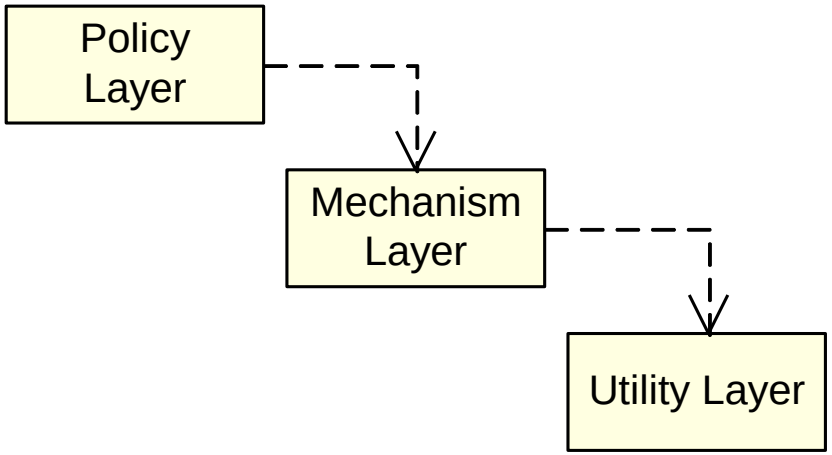
b. Abstractions should not be depended on Implementations. Implementations should be depended on Abstractions.

Modules on higher levels should include general rules of the application and the business logic. Thus, they must have priority and be independent from low level details of implementers. Additionally, these rules are usually useful reusable elements.

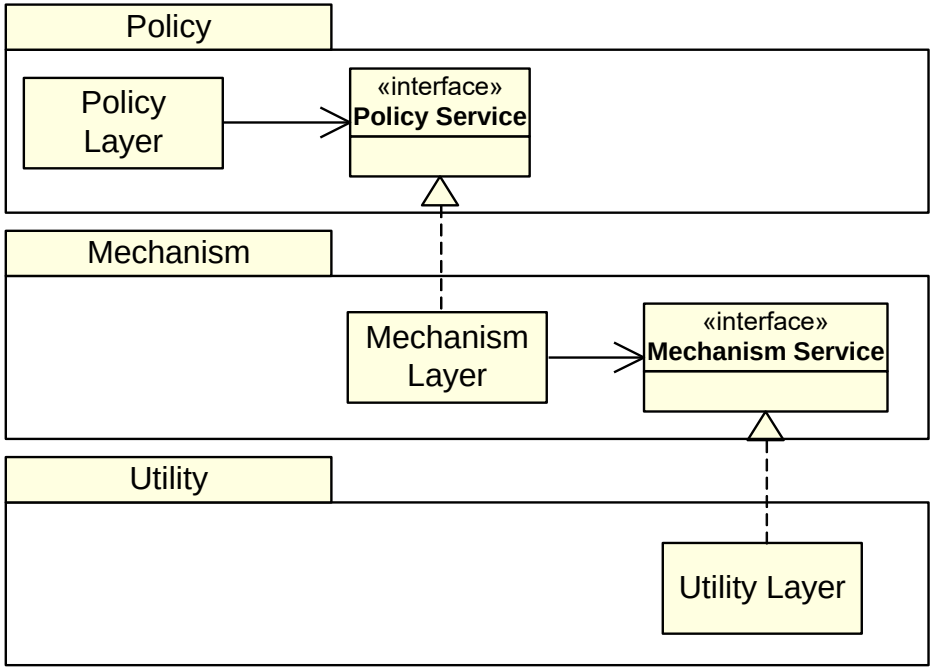
Layering

Booch: A “good” architecture defines clear layers, each layer offers a consistent and coherent set of services through a well-defined interface.

A naïve Approach:



Inversion



Hollywood principle: "Don't call us, we'll call you"

You implement the interfaces, you get registered.
You get called when the time is right.

No class should carry a pointer or reference to a concrete class

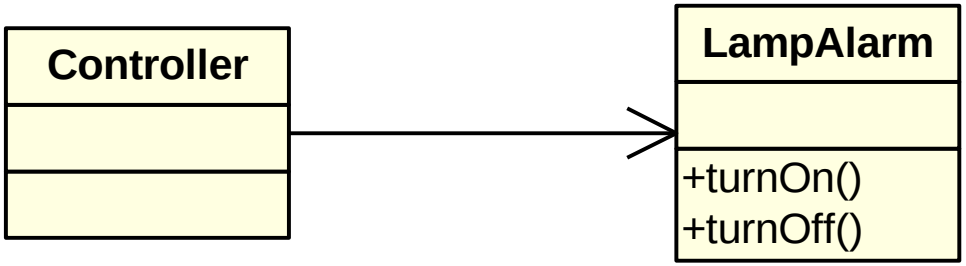
No class should inherit a concrete class

No method should override an implemented method of the parent class

The above rules should not be applied to NOT change prone classes

Example

Suppose a Controller that observes a change in the environment and sends an activation message to an alarm (`LampAlarm`)



Controller "controls" only `LampAlarm` objects

If the source code of `LampAlarm` changes, then the Controller class should be re-checked

If we want to connect the controller with a different mechanism (e.g. an `Engine`) we have to modify the controller

High level policy is not demarcated from implementation

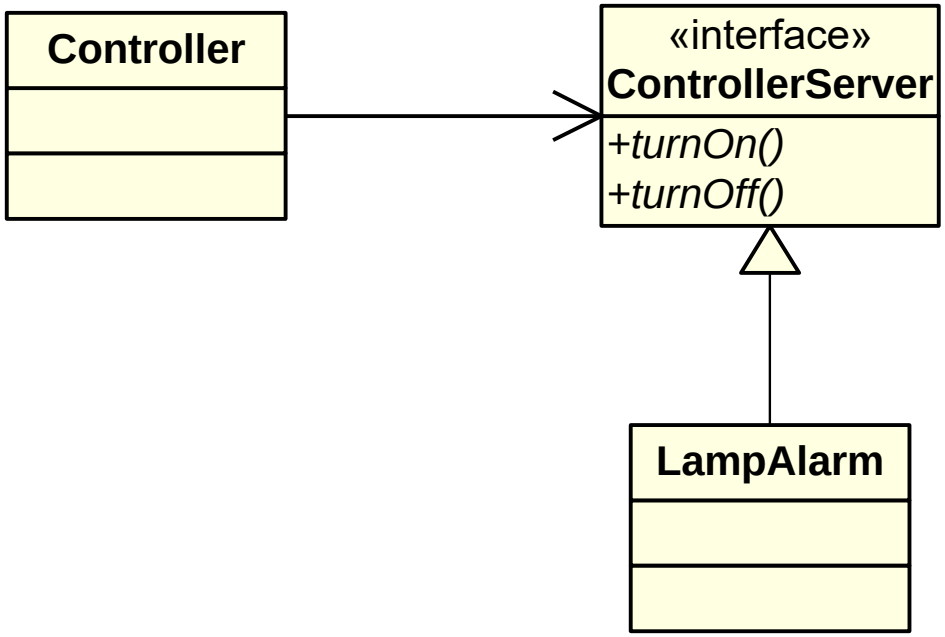
Identify Abstraction and Inverse the Dependency

The implied abstraction is the identification of some change and the sending of an activation message to a recipient

- What is the identification mechanism? Irrelevant (detail)
- Who is the recipient? Irrelevant (detail)

Compliance to DIP: Controller should not depend to `LampAlarm`

Identify Abstraction and Inverse the Dependency



Flexibility: Controller objects can "control" any object that is compatible with the `ControllerServer` interface (even objects that have not yet been invented)

- Functional Programming: Structures in which high-level modules are depended to low-level modules. System architecture is vulnerable from implementation changes

- OOP: Inversed structures

- This difference is considered by many as a real breakthrough of OOP

ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

Ερωτήσεις